SPE-229257-MS

Unlocking Value From Historical Documents: Automated Raster Log Digitization Using Visual Language Models And Computer Vision

Karim Rekik, Ayoub Diouri, and Sohaib Ouzineb, SLB, Montpellier, FRA; Olfa Zened, SLB, Dubai, UAE

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

Abstract

Thousands of historical well logs exist only in raster image or scanned PDF formats, limiting their usability in modern digital workflows. Manually digitizing these logs is time-consuming and error-prone, hindering field evaluation and reservoir characterization efforts. This project introduces an AI-based raster log digitization solution combining computer vision segmentation techniques with the extraction of key metadata from raster logs using Generative AI and Visual Language Models, enabling scalable deep learning digitization with minimal human intervention.

The proposed method leverages cutting-edge multimodal AI models that integrate vision and language understanding to process well log images holistically. Image segmentation techniques are first employed to localize visual components like tracks, headers, and curve line styles. Visual Language Models are then optimized to detect and extract crucial metadata such as log names, units, curve limits, and depth scales by interpreting both text and graphical elements from the segmentation outputs. These combined steps form the foundation for automated log curve digitization using a pre-trained deep learning digitization model.

The system provides accurate and stable results throughout the various stages of the digitization pipeline and significantly reduces the need for manual labeling or user input. By combining generative AI with vision-language learning, the solution improves robustness in handling a wide range of raster log styles, low-quality or handwritten scans. When deployed in cloud environments, this approach enables high-volume, concurrent processing of raster logs—completing in minutes tasks that would traditionally take weeks—and accelerating the ingestion of legacy data into digital ecosystems like OSDU.

Unlike conventional OCR or rule-based methods, this approach understands the context and structure of log images, extracting both visual and textual metadata with precision, as well as understanding complex raster logs structures such as wrapping or single-track multi-logs display. It represents a transformative step in subsurface data management, bringing previously inaccessible historical data into modern interpretation platforms through scalable, intelligent automation.

Introduction

The oil and gas industry maintains extensive archives of historical well logs spanning decades of exploration activities. These legacy documents contain invaluable subsurface measurements critical for reservoir characterization and drilling optimization yet remain trapped in raster image and scanned PDF formats that cannot be integrated into modern digital interpretation workflows.

Current semi-automatic solutions require substantial human intervention, often demanding hours for a few dozens of wells to achieve adequate digitization of raster logs. This creates significant bottlenecks that are both time-intensive and error-prone, particularly when processing poor-quality scans or handwritten annotations, thereby limiting scalability for large-scale digitization projects.

In the current state of the art, a semi-automatic raster digitization method was introduced (Shakeel, 2024). This paper presents a curve segmentation method for raster digitization leveraging deep learning segmentation and digitization models. However, such a method is conditional in the sense that it requires the definition by the user of different key metadata, namely the log track boundaries as well as information found in the track header such as the log name, unit, range, and line style for each curve. Another semi-automatic method (Witte et al., 2020) relies on the user manually selecting pairs of control points throughout the curve to interpolate log values between them.

This paper advances beyond current semi-automatic approaches by presenting a fully automated digitization pipeline that eliminates human oversight requirements. The methodology integrates state-of-the-art computer vision segmentation with Generative AI and Visual Language Models to intelligently process legacy well logs rasters with unprecedented accuracy and efficiency.

We first detail in the Methodology section of this paper the different steps of the raster digitization pipeline, namely image preprocessing, raster segmentation, header parsing, depth track parsing, and finally curve digitization. We then show our experimental setups and corresponding results with the goal of evaluating our automated digitization pipeline.

Methodology

As illustrated in Figure 1, this study comprises five main sequential steps:

1. **Image Preprocessing**: Application of advanced image processing techniques to enhance grid line visibility and prepare raster images for optimal structural element detection.
2. **Raster Segmentation**: Automated image segmentation to isolate log tracks, depth tracks, and log track headers from the complete raster document.
3. **Depth Extraction**: Extraction of depth values from depth tracks, establishing precise depth-per-pixel scaling relationships, and deriving the minimum and maximum depths for the current raster.
4. **Header Metadata Extraction**: Automatic extraction of critical metadata from log track headers, including property names, measurement ranges, units, and line styles using Visual Language Models.
5. **Curve Digitization**: Transformation of log curves into digital values for all identified curves using information derived from the preceding steps

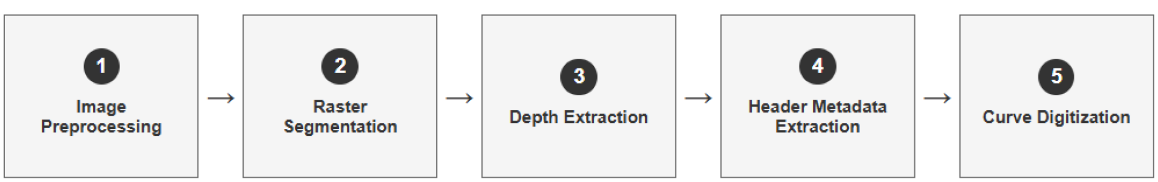

Figure 1—Complete Methodology Overview - Five-Stage Raster Log Digitization Pipeline

Image Preprocessing

This preprocessing step prepares scanned well log images for accurate line detection by cleaning and enhancing the image quality using different functionalities from the OpenCV[1] library, where orientation-specific filters are applied to isolate either vertical or horizontal structural elements. This step is crucial as the pipeline relies on identifying grid lines to segment tracks and detect structural boundaries, and for detecting line styles during the header parsing step.

The following preprocessing steps transform raw scanned documents into optimized formats for structural element detection:

1. Grayscale Conversion: Transform color or multi-channel raster images to single-channel grayscale format. This reduction focuses processing on intensity variations while eliminating color complexities that may interfere with structural element identification.
2. Noise Reduction: Apply Gaussian smoothing to minimize scanning imperfections, document aging effects, and digitization artifacts. This enhancement step creates uniform intensity transitions essential for consistent line detection performance.
3. Binary Transformation: Convert grayscale images to binary format using adaptive thresholding (Otsu, 1979) that adjusts locally to image conditions, ensuring clear separation of grid lines from background even in poorly scanned or degraded documents.
4. Morphological Enhancement: Apply opening and dilation operations to connect fragmented line segments and remove small noise artifacts, ensuring continuous and well-defined structural elements for accurate detection.
5. Selective Line Detection: Apply orientation-specific kernels to enhance either vertical or horizontal line structures based on detection needs.

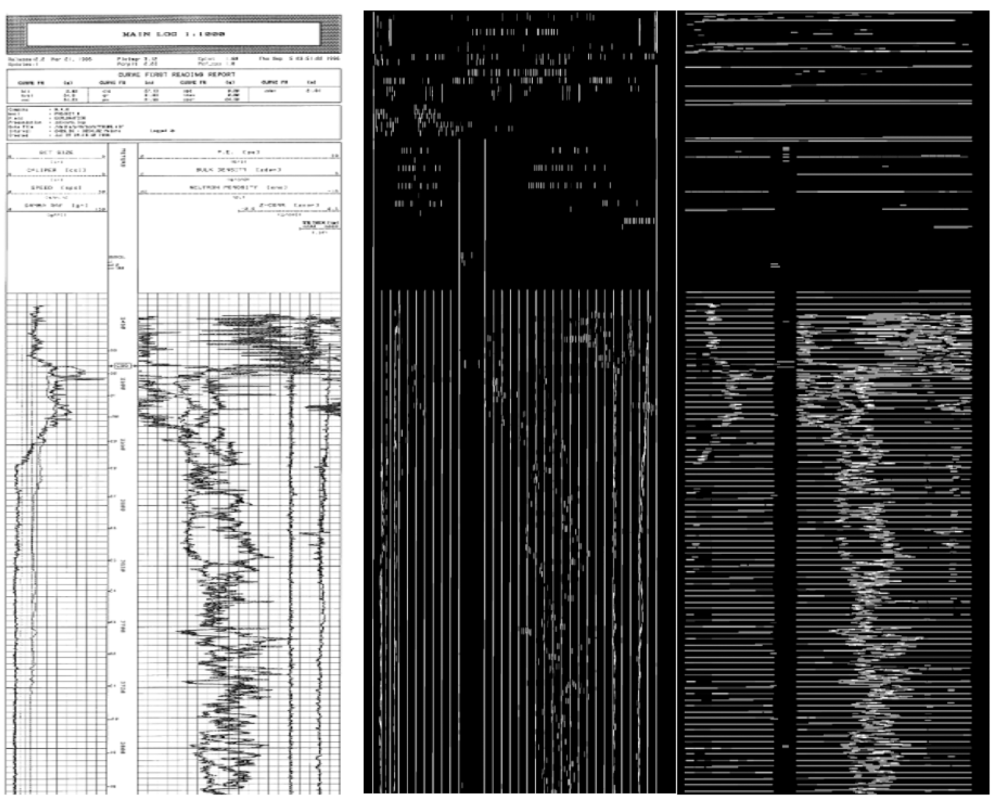

Figure 2—Preprocessing Results for Line Detection Enhancement

This figure demonstrates the preprocessing algorithm applied to a well log raster image. The left panel shows the original scanned document, the middle panel displays the preprocessed image with vertical lines isolated, and the right panel shows the same image with horizontal lines isolated.

Raster Segmentation

Raster segmentation is a computer vision technique that automatically partitions images into distinct, meaningful regions. For well log digitization, this process identifies and isolates three critical components within raster documents: log tracks containing measurement curves, depth track with depth scales, and header sections containing essential metadata such as curve names, units of measurement, and measurement ranges that define the properties and scaling parameters of each logged curve.

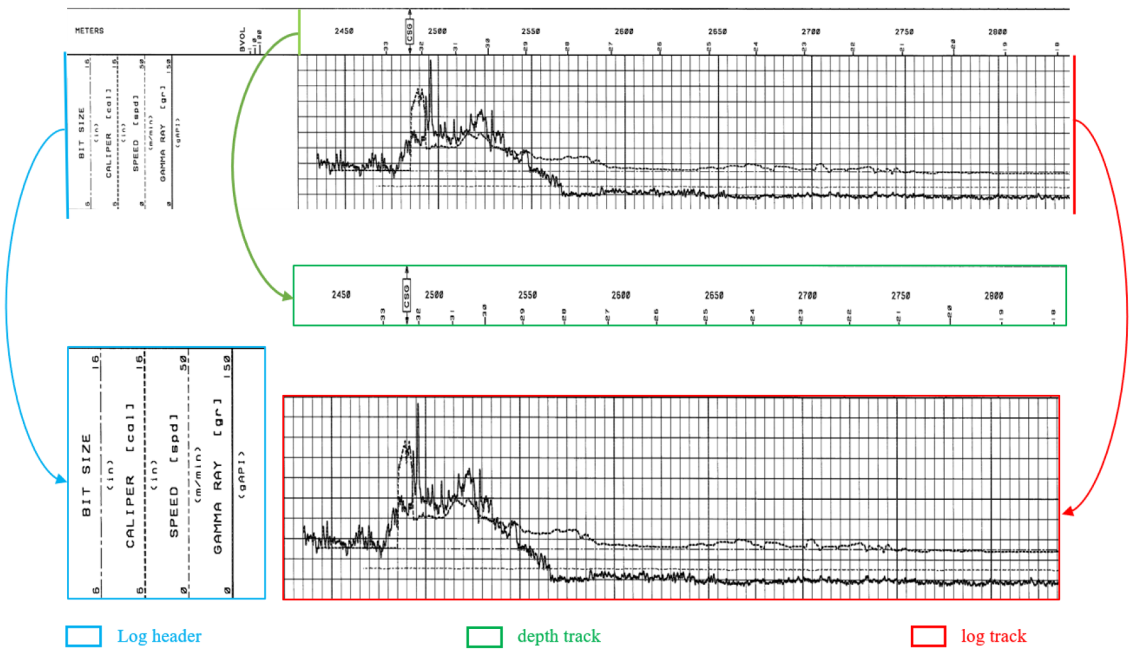

Figure 3—Example of Raster Log Segmentation: Identification of Header, Depth Track, and Log Track

The segmentation process follows a sequential approach where each step builds upon the results of the previous stage. The algorithm progresses through five main stages as illustrated in the flowchart below, ensuring systematic identification and isolation of well log components.

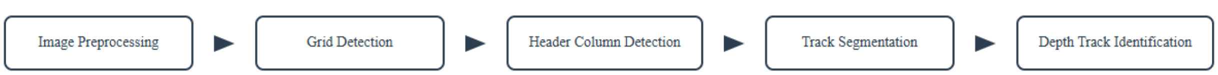

Figure 4—Sequential Raster Log Segmentation Algorithm

Grid Detection

The preprocessing step produces two separate binary images, one containing only vertical lines and another containing only horizontal lines, allowing the algorithm to analyze each orientation independently. From the vertical line image, the algorithm filters detected lines by length, retaining only the longest ones that characterize the actual grid structure. Since well log rasters are typically long documents, the true grid lines are distinguished by their extended vertical span, which allows the process to eliminate shorter noise elements and focus on structural components. These long vertical lines are then categorized based on their positioning, those aligned at the grid's top edge and those aligned at the bottom edge. To locate the grid boundaries, the algorithm analyzes the horizontal line image to find lines that intersect with multiple aligned vertical lines. The top boundary is identified by finding the horizontal line that crosses the greatest number of top-aligned vertical lines, creating a clear separator between the header and the grid. Similarly, the bottom boundary is detected by identifying the horizontal line that intersects with the most bottom-aligned vertical lines, establishing the grid's lower limit completing the grid segmentation.

Header Column Detection

The algorithm operates under the assumption that the header section is positioned directly above the grid with no gap between them, which simplifies the detection process to identify where the header content begins. With the grid separator line already established, the remaining task is to determine the vertical starting position of the structured header information. The algorithm processes the vertical line image to detect lines within the header area that terminate near the grid separator line, indicating they serve as column dividers rather than decorative elements. These boundary candidates are filtered to exclude lines too close to image edges and analyzed for their alignment patterns to pinpoint the vertical position where column-based information starts. Once this starting position is identified, the process extracts the horizontal positions of vertical separators that define individual track columns within the header section, completing the header column detection.

Track Segmentation

This segmentation step extends header column boundaries into the log curves section using robust line propagation techniques. Rather than simple vertical projection, the algorithm searches for actual vertical lines near each header column boundary position within the log grid area. When direct line matches are found, they are used as precise track separators. For boundaries where no suitable lines exist due to scanning artifacts or image quality issues, the process employs line translation, taking reference lines from successfully detected boundaries and translating them to the required positions. This approach ensures robust track separation even when individual vertical lines are discontinuous or non-vertical due to scanning distortions, creating accurate column boundaries that follow the actual document structure rather than assumed straight projections. Once tracks are segmented, the depth track is identified as the track with the smallest width, completing the track segmentation.

Depth Extraction

The depth track represents a narrow vertical column within the well log that contains numerical depth values indicating the measured depth intervals along the wellbore (as shown in Figure 5). Unlike other tracks that display continuous curves, the depth track typically contains only sparse depth annotations, usually just a few numerical values distributed vertically throughout the entire log section.

For successful curve digitization, the algorithm must determine the precise depth corresponding to every pixel row in the log curves, since each digitized measurement value must be mapped to its exact subsurface depth position. This requires extraction of both the minimum and maximum depth values that define the grid's vertical span. The first depth represents the shallowest measurement point where the grid begins, while the last depth corresponds to the deepest measurement point where the grid ends. However, the actual first written depth value in the track often appears several pixels below the grid's starting line (represented by the red arrow in Figure 5), and similarly the last written depth value typically appears several pixels above the grid's ending line (represented by the green arrow in Figure 5), creating gaps that must be accounted for through depth-per-pixel calculations. The algorithm must therefore extract the sparse depth annotations, calculate the depth-per-pixel scaling relationship, to estimate the start depth and last depth values.

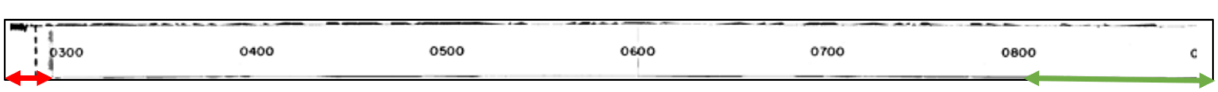

Figure 5—Depth Track Illustration Showing Depth Annotation Gaps

Red arrow indicates the gap between grid start line and first written depth value (0300).
Green arrow indicates the gap between last written depth value (0800) and grid end line.

The pipeline proposed to determine the first and last depth values operates through four sequential processing stages. The system first segments the depth track into manageable sections by identifying blank regions, then extracts depth numbers using Optical Character Recognition (OCR) combined with vision language model (VLM) verification for accuracy. Multiple depth-per-pixel (DPP) calculations are performed using consecutive depth value pairs. This automated approach reliably delivers the essential start and end depth boundaries required for precise curve digitization.

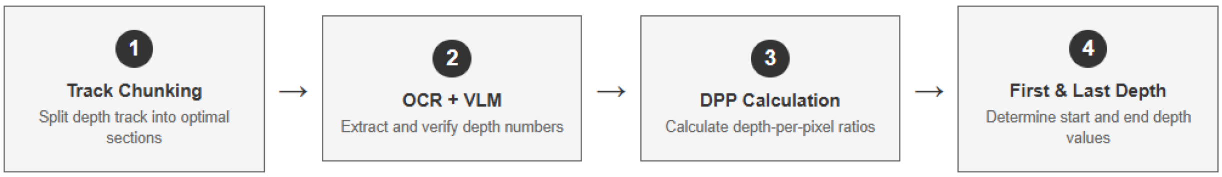

Figure 6—Depth Track Parsing Pipeline Overview

Track Chunking for OCR Optimization

The depth track segmentation process is implemented to optimize OCR performance with the Tesseract library (Smith, 2007), which experiences significant accuracy degradation when processing long vertical documents. Rather than applying OCR to the entire depth track as a single image, the algorithm strategically segments the track into manageable sections to maintain high text recognition accuracy, this chunking is performed by identifying substantial blank regions as natural cutting points, this approach prevents cutting through depth annotations and ensures complete depth number detection across the entire track length.

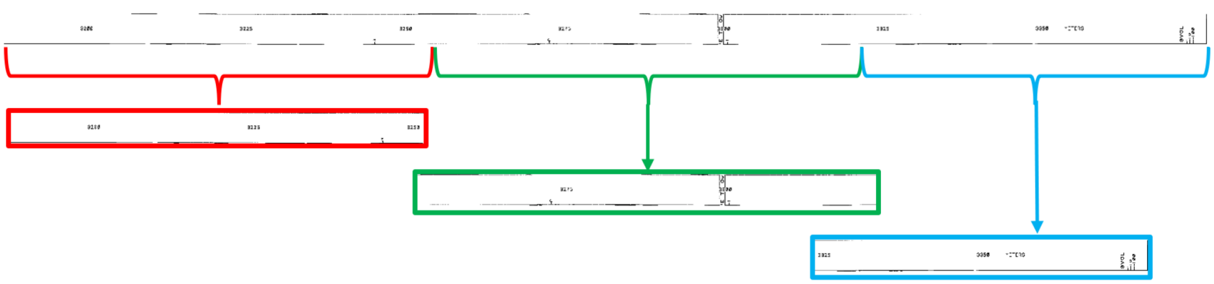

Figure 7—Example of Adaptive Segmentation Applied to Long Depth Track Document

OCR-VLM Hybrid Depth Extraction

The depth number extraction employs a hybrid approach combining Tesseract OCR with Visual Language Models to leverage the strengths of both technologies. Each segmented depth track section is processed individually, with Tesseract OCR utilized primarily for its reliable bounding box detection capabilities rather than text accuracy, while traditional OCR libraries often misread depth values, they consistently identify the spatial locations where text appears within the image segment. This positional information is crucial for subsequent depth-per-pixel calculations, making classical OCR approaches valuable for spatial detection tasks.

The detected bounding boxes are used to crop individual depth value regions from the original image, creating focused image patches that are then processed by Visual Language Models. VLMs excel at single-task text extraction from cropped images, providing significantly higher accuracy for reading the actual depth values compared to traditional OCR. Rather than using VLMs directly on the entire depth track, this hybrid approach addresses the need for precise positional data while maintaining extraction accuracy. To optimize processing efficiency, all cropped images are processed in batch mode using GPU acceleration through the VLLM library (Kwon et al., 2023), allowing parallel inference across multiple depth value regions simultaneously.

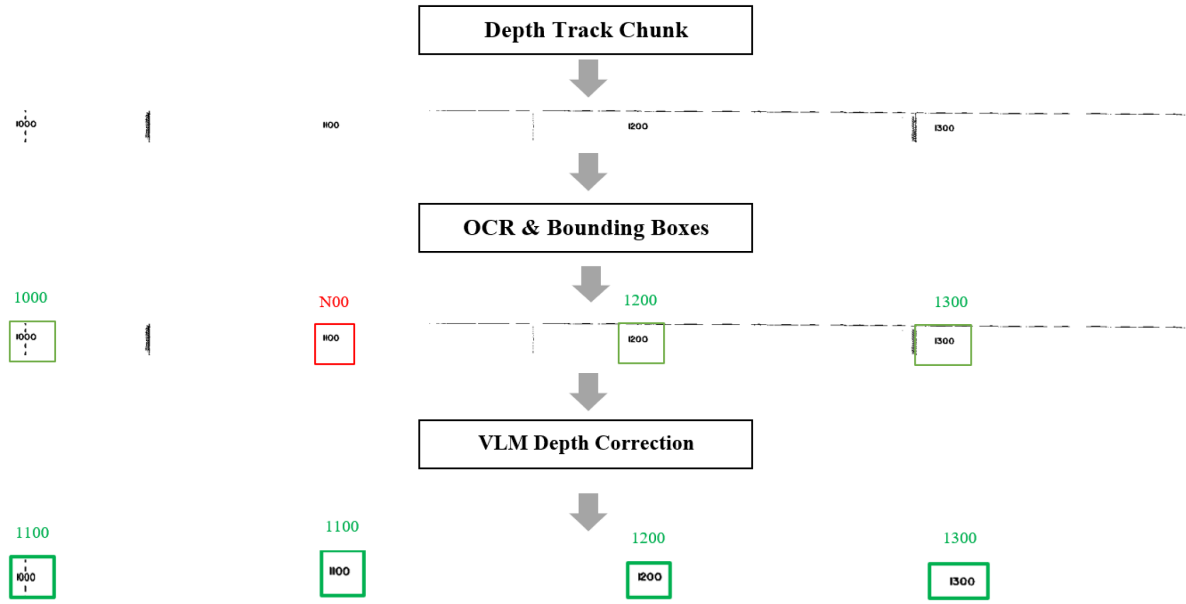

Figure 8—OCR-VLM Hybrid Depth Extraction Pipeline

This hybrid methodology represents a novel approach to document text extraction, as most current VLMs are not designed for text detection with bounding box localization but rather excel at direct text extraction from pre-defined regions. By combining the spatial detection capabilities of classical OCR with the superior text recognition accuracy of modern VLMs, this algorithm creates a more robust extraction system that overcomes the individual limitations of each technology while maximizing their respective strengths.

Depth-Per-Pixel Calculation and Robust Estimation

Following depth extraction, the algorithm first filters the verified results to retain only valid numeric values, eliminating any non-numeric text. The depth-per-pixel (DPP) scaling factor is then calculated using the fundamental relationship between consecutive depth annotations and their corresponding pixel positions:

$$\mathbf{DPP} = (\mathbf{Depth_2} - \mathbf{Depth_1}) \,/\, (\mathbf{Position_2} - \mathbf{Position_1})$$

To enhance algorithmic robustness, the system calculates DPP values not only between consecutive depth pairs (i and i-1) but also extended pairs (i+1 and i-1), generating multiple scaling estimates across the entire depth sequence. This redundant calculation approach ensures that even if individual depth values contain errors or if certain pairs produce inaccurate calculations, the algorithm can recover the true scaling relationship. The median value is selected from this expanded set of DPP calculations, as it represents the most frequent and statistically reliable estimate while being resistant to outliers that might result from OCR errors or anomalous depth annotations.

Header Metadata Extraction

The header parsing process employs a two-step approach combining Florence OCR (Xiao et al, 2023) with Visual Language Models to extract structured metadata from log track headers. Florence OCR provides initial text detection and spatial positioning, while VLM handles the complex interpretation of header structure, including curve names, units, measurement ranges, and hierarchical relationships between elements. The positional information from Florence OCR enables the algorithm to establish spatial relationships between numeric range values and their corresponding horizontal line patterns beneath them. The algorithm uses detected horizontal lines to segment individual line style patterns for each curve, processing these visual elements to extract precise styling information required for digitization. Figure 9 shows the above described metadata extraction and line style extraction steps.

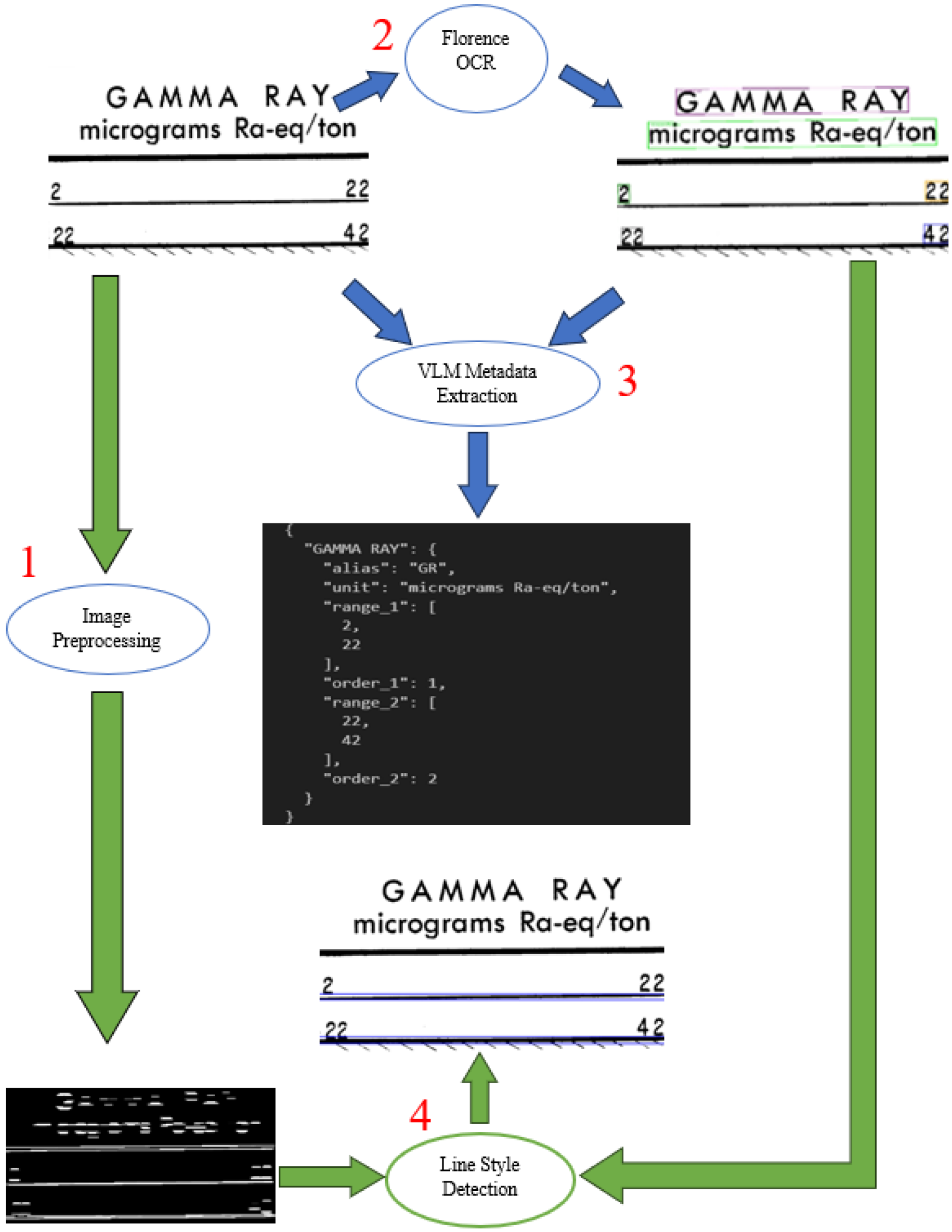

Figure 9—Header Parsing Pipeline for Metadata and Line Style Extraction. (Steps 2 and 3 represent the OCR-VLM metadata extraction, steps 1 and 4 represent line style extraction)

OCR-VLM Metadata Extraction

To achieve optimal metadata extraction accuracy, the system employs an agentic processing approach inspired by multi-agent workflows where complex tasks are decomposed into specialized subtasks. As shown in Figure 9, the pipeline utilizes Florence OCR as the foundational backbone for spatial detection, providing structured text that preserves the original layout from the header image. The first VLM agent in the metadata extraction step receives both the original image and the spatially structured OCR text, focusing exclusively on curve extraction to identify curve names, units, and ranges without the complexity of spatial ordering. A second VLM agent then processes the original image alongside the extracted metadata from the first agent, applying global sequential ordering based on visual layout analysis. This agent-based decomposition aligns with the principle that specialized agents perform better on focused tasks than generalist models handling multiple complex requirements simultaneously. By separating extraction from ordering, each VLM operation can optimize for its specific cognitive demand, resulting in more reliable and accurate metadata structuring compared to single-prompt approaches that attempt to handle all extraction complexities simultaneously.

Figure 10 demonstrates an extraction example where curve names, units, and ranges are directly visible in the image, but the global ordering is deducted by the VLM through spatial analysis. This ordering information serves as the critical link for line style matching, enabling the algorithm to correlate each extracted range with its corresponding horizontal line style based on sequential position within the header structure.

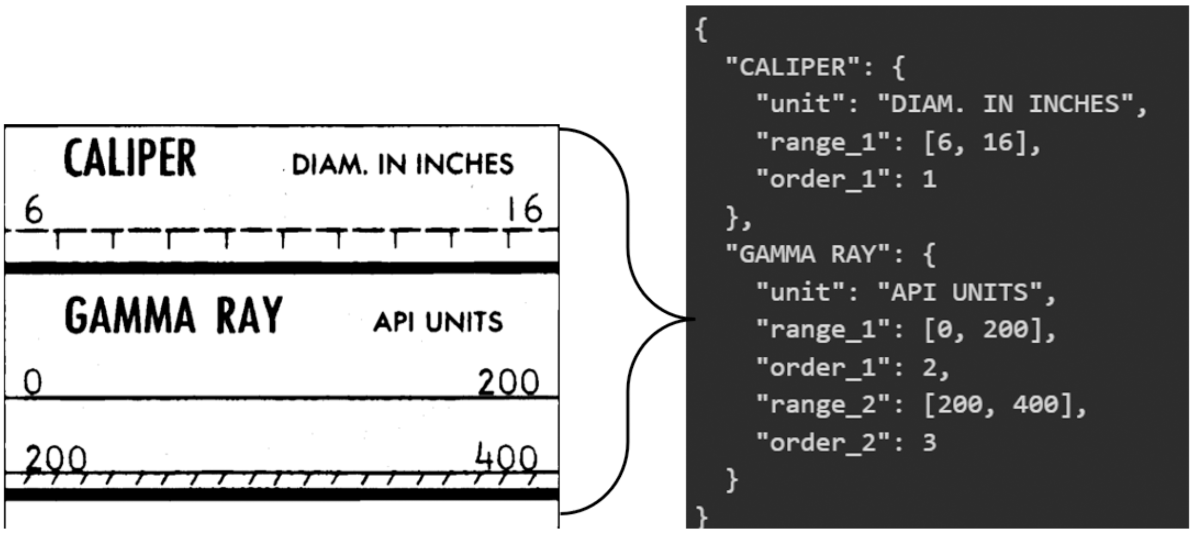

Figure 10—VLM Metadata Extraction Demonstrating

Line Style Extraction

The line style extraction process preprocesses the header image to detect horizontal line segments, filtering short artifacts. Using numeric text positions from Florence OCR, the algorithm groups text regions into range pairs based on horizontal alignment, then matches each range pair to its closest horizontal line pattern using vertical distance calculations. This spatial matching filters out horizontal lines that are not associated with measurement ranges, retaining only line styles that correspond to actual curve definitions for digitization.

Figure 11 shows an example of the detection results where the blue bounding boxes highlight the extracted line styles that correspond to measurement ranges.

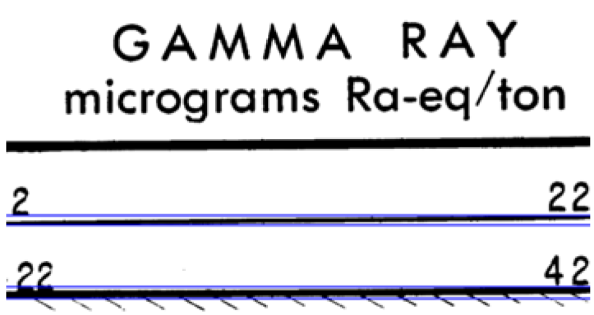

Figure 11—Line Style Extraction Results

Curve digitization

The curve digitization process employs a pre-trained model (Shakeel, 2024) to perform conditional curve segmentation, where individual curves are isolated from multi-curve log tracks based on line style specifications extracted from header parsing. The log track is divided into square tiles, with each tile augmented by pasting the target line style image along all four sides to provide the segmentation model with the necessary conditional information for accurate curve identification. A three-layer fully convolutional neural network then converts the segmented curves into digital CSV format through row-level classification, where each row of the segmented curve image is classified to determine the curve's value at its vertical position. Figure 12 shows the result of the digitization process on an original scanned log track.

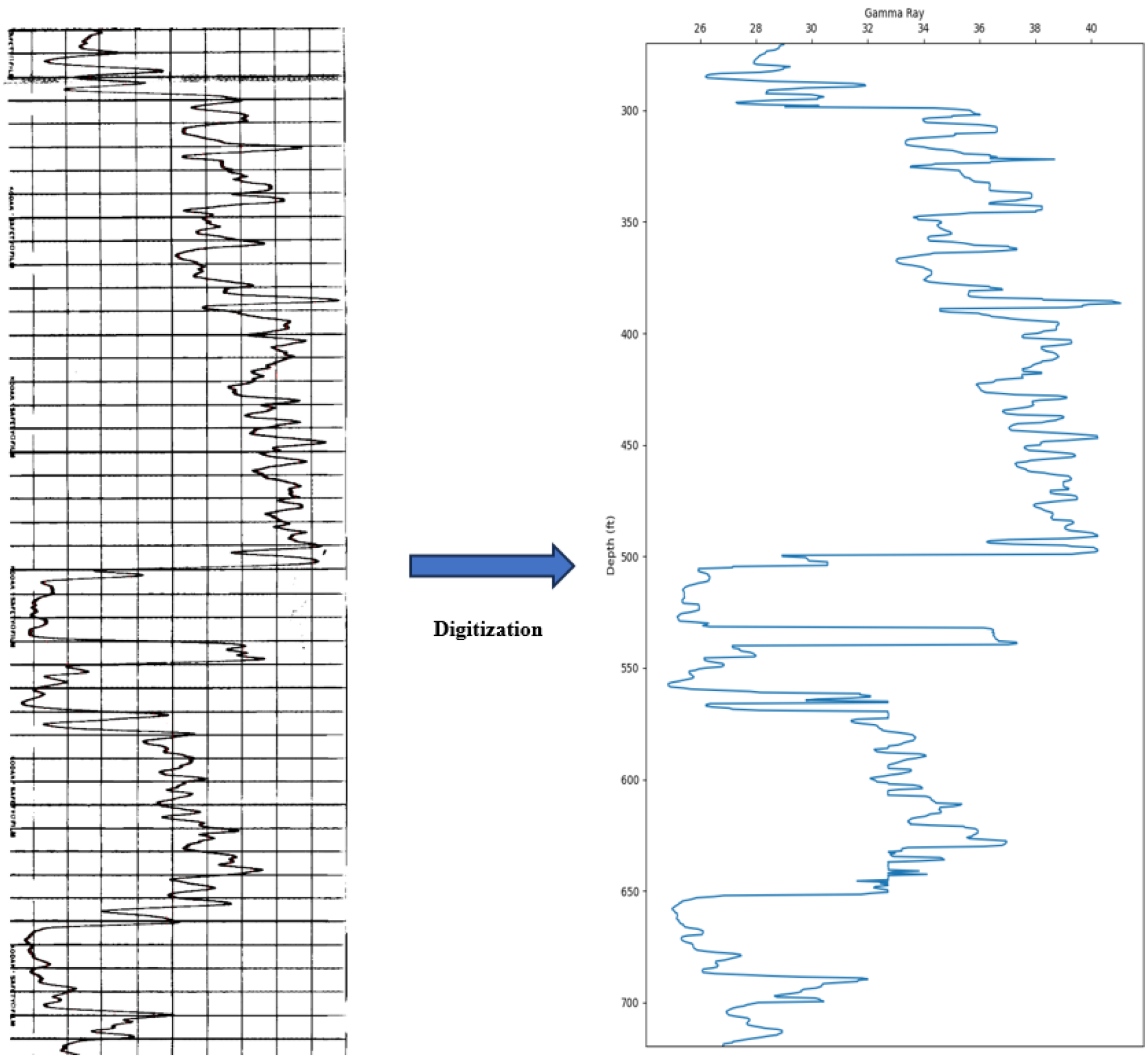

Figure 12—Final Digitization Results Showing Transformation from Paper Log (left) to Digital Data (right)

Experiments

We first present in this section the different setups of the experiments we run to evaluate the different intermediate steps of our digitization pipeline. We then present the results and analyze them.

Experiment Raster Data

The experimental evaluation utilizes a diverse collection of 15 raster well log data styles represented by 15 documents each corresponding to a different raster type and style, specifically covering single-track configurations, multi-track layouts, varying header formats and diverse measurement unit systems to validate algorithm robustness across the full spectrum of legacy well log documentation found in industry archives sourced from multiple geographic regions to ensure comprehensive algorithm testing. The dataset includes public raster logs from Colorado and North Dakota in the United States, providing examples of American petroleum industry documentation standards, alongside Netherlands public well logs representing European formatting conventions **(Fig. 13)**.

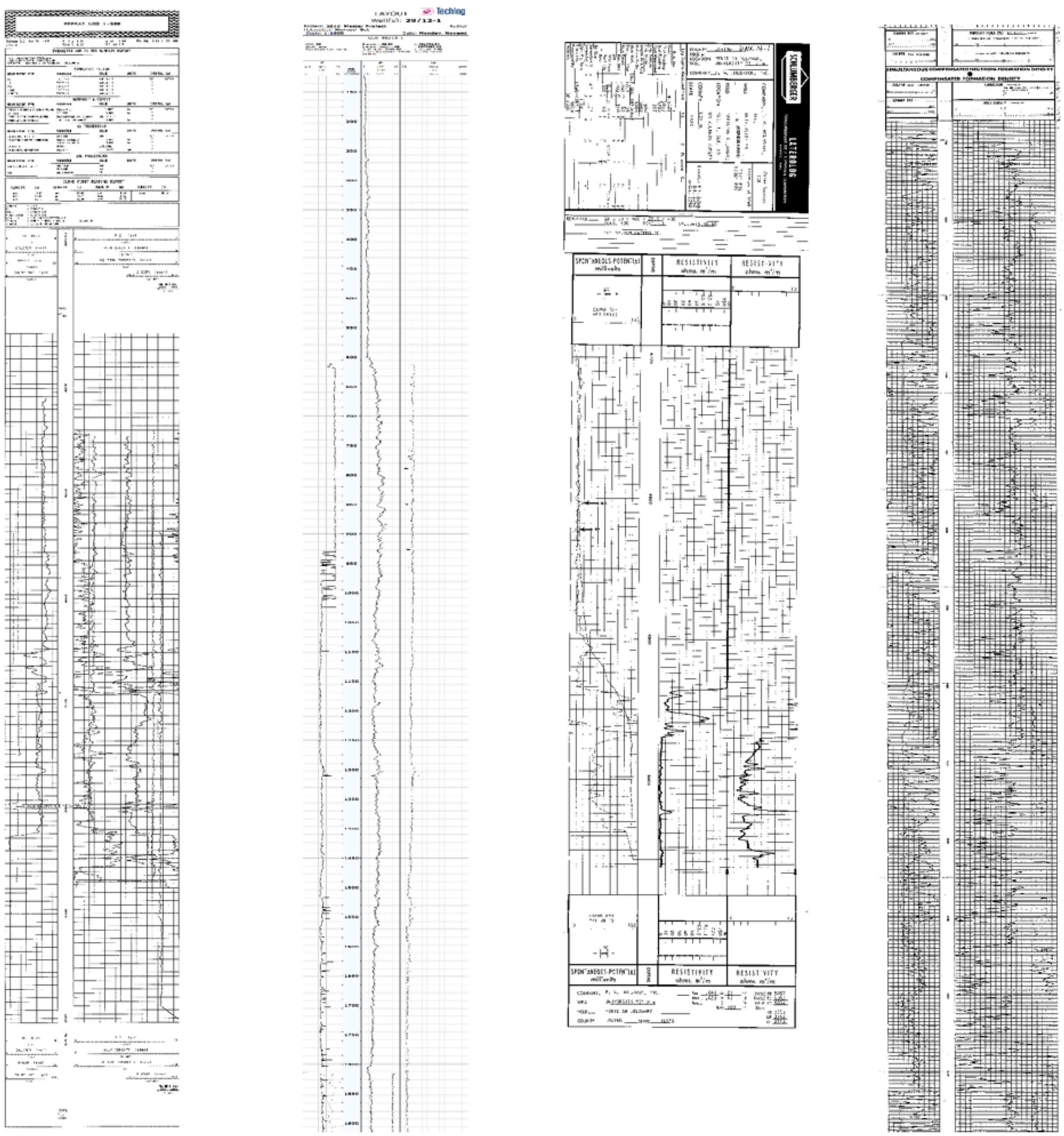

Figure 13—Examples Of Diverse Raster Styles in The Test Dataset

Experimental Setup

Experimental Setup – Track Segmentation Evaluation. The segmentation performance is evaluated using Intersection over Union (IoU) to assess track boundary detection accuracy. IoU measures the overlap ratio between predicted and ground truth bounding boxes using the formula:

$$\mathbf{IoU} = (\mathbf{Area\ of\ Intersection}) / (\mathbf{Area of Union})$$

IoU values range from 0 (no overlap) to 1 (perfect match). Ground truth bounding boxes have been manually picked for our test set of raster documents.

Experimental Setup - Depth Extraction Evaluation. The depth parsing performance is evaluated using Median Absolute Error (MAE) to assess the accuracy of start and end depth estimation. For each depth track image, the algorithm extracts predicted start and end depth values which are compared against manually annotated ground truth values.

The evaluation metric is calculated using the following formula:

$$\mathbf{MAE} = \mathbf{median}(\mathbf{predicted\_depth} - \mathbf{ground\_truth\_depth}|)$$

Where the absolute difference is computed separately for start depths and end depths across all test images. Ground truth start and end depths are computed manually for each test raster document using the displayed depth values and corresponding depth ticks.

Experimental Setup - Header Metadata Extraction Evaluation. The header parsing performance is evaluated using precision and recall metrics to assess the accuracy of curve name, unit, and range extraction. Each extracted metadata field is compared against manually annotated ground truth using similarity thresholds and exact matching criteria. The precision metric allows us to answer the question: "From my extracted fields, how many are correct?", while the recall metric allows us to answer the question: "From the expected ground truth fields, how many was I able to successfully extract?". The evaluation metrics are calculated using the following formulas:

$$\mathbf{Precision} = \#(\mathbf{Correctly\ Extracted\ Fields}) / \#(\mathbf{Total\ Extracted\ Fields})$$

$$\mathbf{Recall} = \#(\mathbf{Correctly\ Extracted\ Fields}) / \#(\mathbf{Total\ Ground\ Truth\ Fields})$$

Name matching uses the Ratcliff/Obershelp algorithm (Ratcliff & Metzener, 1988) for computing the string distance similarity between extracted text fields and ground truth text fields (like curve name) with a threshold of 0.8. Unit similarity applies the same 0.8 threshold after normalization, and range matching requires exact correspondence within a tolerance of 0.01

As shown in Figure 14, the manually extracted ground truth contains 12 total fields (4 curve names, 4 units, and 4 ranges). The Gemma-3-12B VLM successfully extracted all four curve names and units with perfect accuracy, but failed on range extraction for CALIPER and BIT SIZE, incorrectly identifying both ranges as [0, 20] instead of [10, 20]. This resulted in 10 correctly extracted fields out of 12 total fields. For this specific example, both precision and recall scores equal 83.3% (10/12).

Four Visual Language Models were evaluated based on industrial deployment requirements: Gemma-3-12B (Kamath et al, 2025), Pixtral-12B (Agrawal et al., 2024), Mistral-Small-3.1-24B-Instruct, and Mistral-Small-3.2-24B-Instruct. We chose these models on the following criteria: open-weight models with permissive licensing for commercial use, on-premise deployment capability for data sovereignty, and optimal accuracy within computational constraints. The 12B parameter models (Gemma-3-12B and Pixtral-12B) enable deployment on our local NVIDIA A6000 GPUs with 48GB VRAM, while the 24B Mistral models were deployed on a cloud infrastructure but offer potential accuracy improvements. This evaluation strategy aimed at finding the smallest models capable of delivering optimal accuracy for metadata extraction while maintaining practical deployment constraints for production environments.

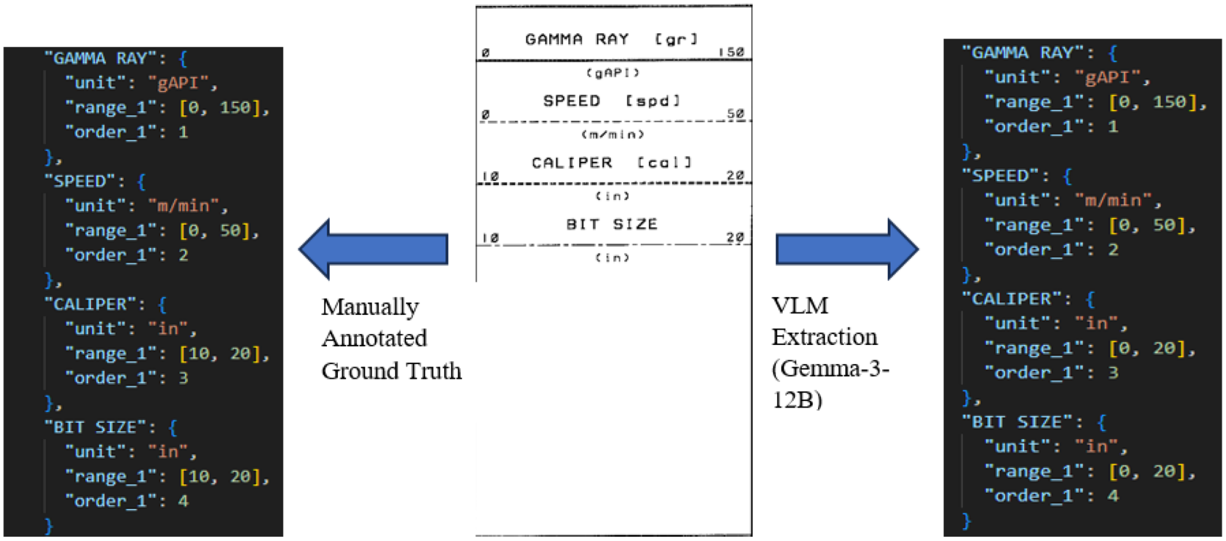

Figure 14—Comparison of Manual Annotation vs. VLM Extraction Results for Header Metadata

We employed these pre-trained models using several prompt engineering techniques with structured output formatting and context-aware instructions, decomposing the extraction task into two specialized steps (metadata extraction followed by spatial ordering), and augmenting VLM processing with OCR-derived spatial information to improve accuracy. These methods allow us to optimize extraction performance without requiring model modifications. For future work, we plan to fine-tune our visual language models using knowledge distillation on well log header datasets to improve domain-specific extraction accuracy.

Experimental Setup - Line Style Extraction Evaluation. The line style extraction performance is evaluated using the same IoU metric as segmentation evaluation

Results

Track Segmentation

The experimental evaluation demonstrates strong performance across all pipeline components. The segmentation module achieved a mean **IoU of 0.937**, indicating highly accurate log track boundary detection with their log track headers successfully separated from curve regions.

Depth Extraction

The depth parsing pipeline achieved a **median absolute error of 3.94** units, meaning the algorithm accurately detects depth boundaries within approximately 4 depth units or less for most test cases. This precision is sufficient for establishing reliable depth-per-pixel scaling relationships required for accurate curve digitization.

Metadata Extraction

The header parsing evaluation compared four Visual Language Models: Gemma-3-12B, Pixtral-12B, and two versions of Mistral-Small (3.1-24B-Instruct and 3.2-24B-Instruct), as shown in Table 1 where we report all extraction metrics, precision and recall, on the different metadata fields and in total.

Table 1—Metadata Extraction Results Table For Different Visual Language Models

Model name	Precision				Recall			
	Name	Range	Unit	Total	Name	Range	Unit	Total
Gemma-3-12B	90.9%	86.0%	84.8%	87.2%	90.9%	86.0%	84.8%	87.2%
Pixtral-12B	81.0%	80.0%	61.9%	74.6%	61.9%	57.1%	44.8%	53.8%
Mistral-Small-3.1-24B-Instruct	**96.9%**	**95.3%**	**96.9%**	**96.3%**	**96.9%**	**95.3%**	**96.9%**	**96.3%**
Mistral-Small-3.2-24B-Instruct	**96.9%**	**95.3%**	**96.9%**	**96.3%**	**96.9%**	**95.3%**	**96.9%**	**96.3%**

Both Mistral-Small models achieved the best performance among all tested models with precision and recall scores of 96.9% for names, 95.3% for ranges, and 96.9% for units, resulting in overall precision and recall of 96.3% for both metrics. Metrics for both Mistral-Small models are reported in Figure 15.

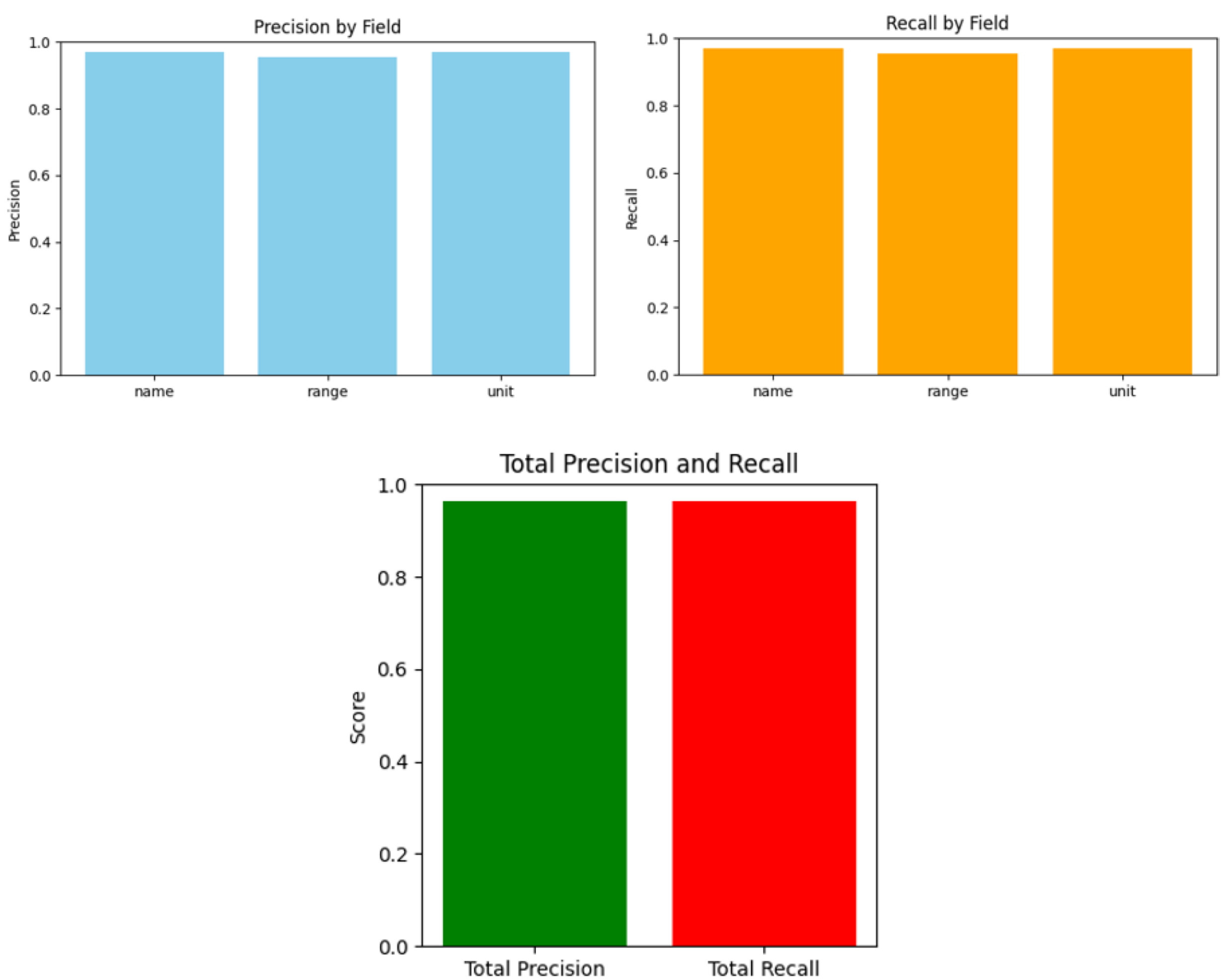

Figure 15—Mistral-Small Models Header Metadata Extraction Evaluation Metrics For Each Field And In Total

Gemma-3-12B showed solid performance with precision scores of 90.9% for names, 86.0% for ranges, and 84.8% for units, with corresponding recall values of 90.9%, 86.0%, and 84.8% respectively, resulting in overall precision and recall of 87.2% for both metrics. Metrics for the Gemma-3-12B model are reported in Figure 16.

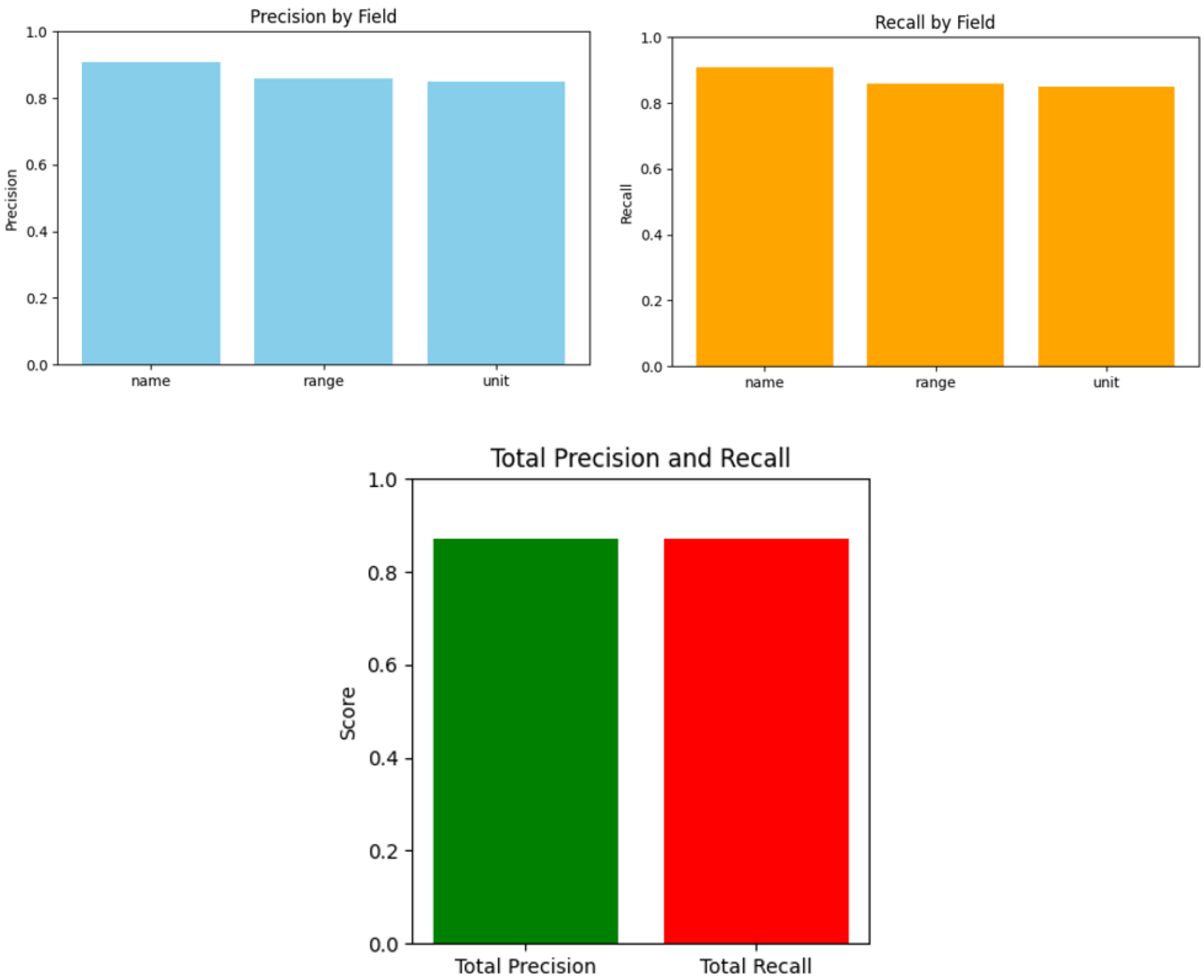

Figure 16—Gemma-3-12B Header Metadata Extraction Evaluation Metrics For Each Field And In Total

Pixtral-12B achieved precision scores of 81.0% for names, 80.0% for ranges, and 61.9% for units, with corresponding recall values of 58.6%, 57.1%, and 44.8% respectively, resulting in overall precision and recall of 74.6% and 53.8%. Metrics for the Pixtral-12B model are reported in Figure 17.

We observe identical precision and recall scores for both Mistral-Small models (96.3%) and Gemma-3-12B (87.2%). This equality occurs because these models correctly identify the expected header metadata JSON fields keys without adding or missing field keys, resulting in perfect JSON schema keys similarity. This reflects the models' reliability in following structured output requirements. As the ground truth schema is similar to the extracted schema, we have:

$$\#(\textbf{Total Extracted Fields}) = \#(\textbf{Total Ground Truth Fields})$$

which implies similar values for both precision and recalls, with errors coming only from differences in extracted values but not from schema keys.

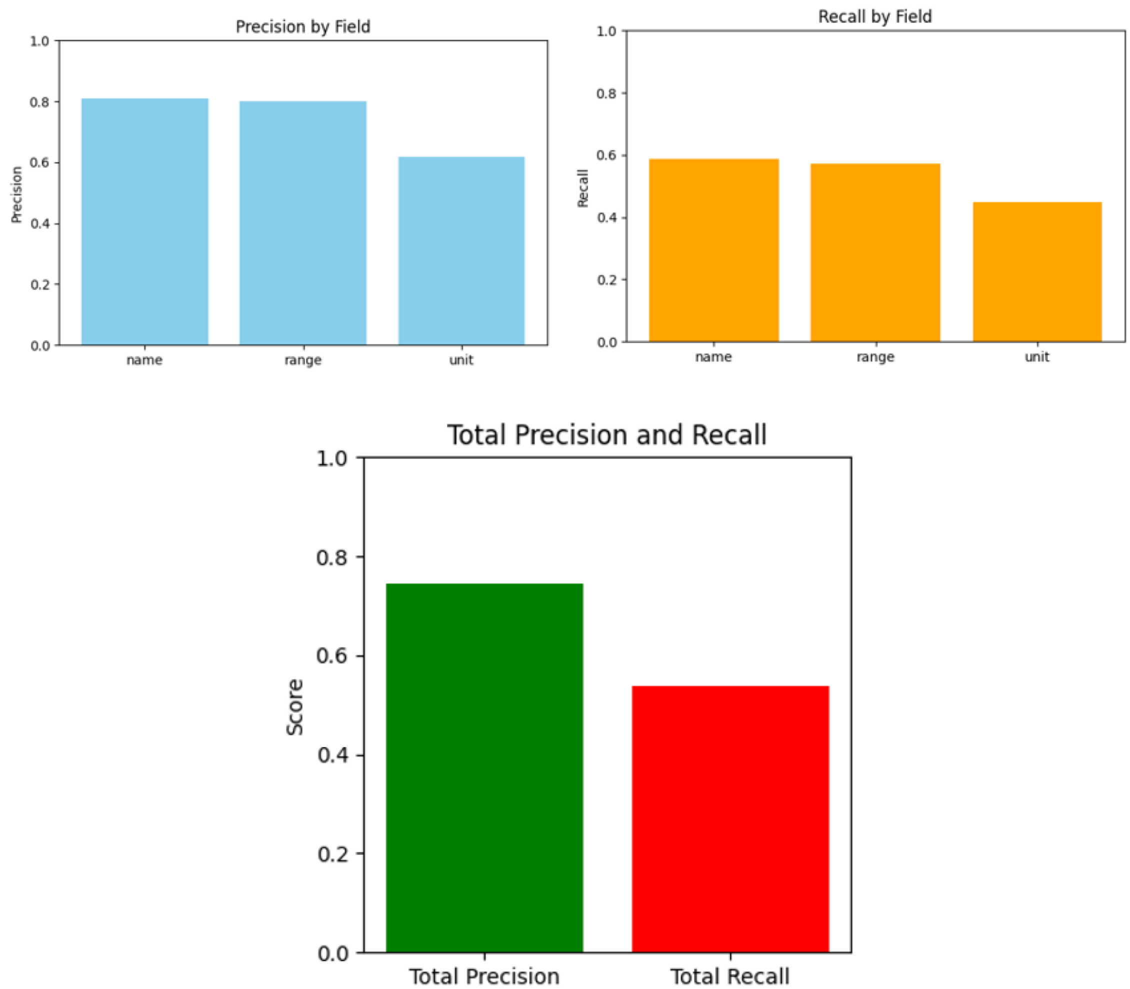

Figure 17—Pixtral-12B Header Metadata Extraction Evaluation Metrics For Each Field And In Total

The Mistral-Small models achieved superior performance compared to both Gemma-3-12B and Pixtral-12B, which is expected given their larger parameter count (24B vs 12B). Both Mistral-Small variants achieved identical results of 96.3% precision and recall because both models encountered the same challenging header format shown in Figure 20 in the results discussion section, which prevented them from achieving perfect scores.

For the models with 12B parameters, Pixtral-12B results are below those of Gemma-3-12B for all header metadata fields, with notably lower recall metrics due to failures in returning valid JSON outputs with the requested schema for the Pixtral-12B model. This performance gap stems from fundamental training differences:

Gemma-3-12B, being a more recent model, was specifically tuned during post-training for enhanced instruction-following capabilities and native support for structured outputs, including JSON schema adherence (Kamath et al, 2025).

In contrast, Pixtral-12B's training focused more on vision-text understanding rather than structured data generation. When prompted for JSON output, Pixtral-12B produces malformed responses with incorrect bracket placement, missing commas, or incomplete object structures that cannot be parsed by downstream components (Agrawal et al., 2024).

This shows that models with the same number of parameters can behave differently, highlighting the need for regular and exhaustive visual language model benchmarking.

Line Style Extraction

Finally, the line style extraction component achieved the highest performance with a **mean IoU of 0.969**, indicating near perfect line style extraction.

Results Discussion

Performance variations in our digitization pipeline stem from raster logs that deviate from standard industry formatting conventions. These non-conventional documents challenge our algorithm's design assumptions and create processing difficulties at different stages of the pipeline.

Documents like the one shown in Figure 18 illustrate this challenge, the header section lacks the vertical column separators that our algorithm expects to find. Our segmentation approach relies on vertical dividing lines within the header to determine individual track positions, so without these visual separators, the system cannot establish where one track ends and another begins, resulting in merged tracks or incorrectly positioned boundaries that affect all downstream processing steps.

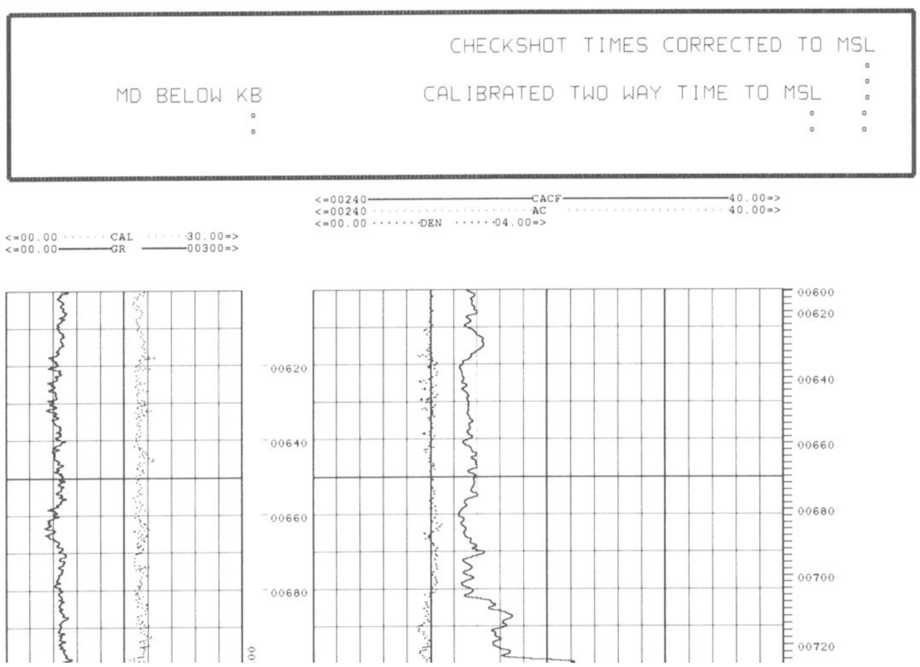

Figure 18—Header Section Lacking Clear Column Separators

Additionally, a separate issue affects the final digitization step: some logs use non-standard line style representations, such as double-headed arrows instead of simple horizontal lines, cannot be recognized by our detection system (demonstrated in Figure 19).

Additionally, headers with atypical layouts like the one in Figure 20 present challenges for Visual Language Models during metadata extraction. All VLMs we tested struggled to correctly extract metadata from this header because it contains multiple ranges presented in an unconventional format that makes it difficult to assign which range corresponds to each curve.

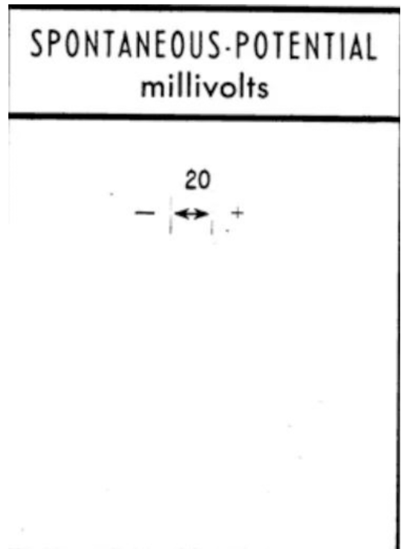

Figure 19—Example of Gap Between Header and Grid Sections

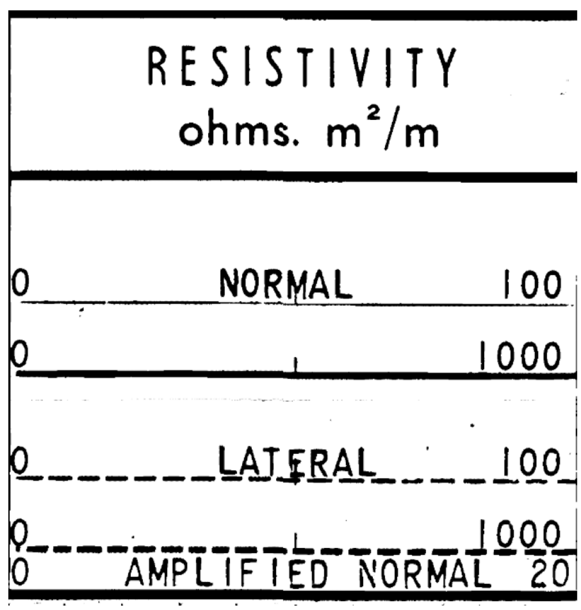

Figure 20—Atypical Header Format with Multiple Overlapping Ranges

Conclusion

In this work, we introduced a novel fully automated raster log digitization method which can take an entire well log raster document as input and return a digitized version of each displayed log as output. This marks an important shift from the state of the art in raster digitization which, despite automating many intermediate steps, still requires some human intervention, making the digitization process cumbersome when processing a large batch of raster documents. Our solution leverages state-of-the-art computer vision and generative ai visual language model technologies to do automatically in sequence: image preprocessing, raster segmentation, depth extraction, header metadata extraction, and curve digitization. Experiments show accurate results throughout all intermediate steps of the digitization pipeline, with segmentation tasks both performing with an IoU greater than 93%, the depth extraction being precise with an error margin of 4 feet, the header metadata extraction scoring total precisions and recalls of 87.2% for the Gemma-3 12B model and 96.3% for the Mistral-Small 24B models, and the digitization step correctly extracting more than 80% of the curve for 72% of all curves in the test dataset (Shakeel, 2024). These results also show our ability to process different styles and formats of raster documents, accounting for single-track multi-logs display, wrapping, and possible multiple ranges and different line styles for the same log. By processing multiple raster documents in parallel using different parallelization technologies such as multi-threading and LLM batching with VLLM, we can run this pipeline at scale. The automatic, scalable, and accurate properties of our pipeline allow us to deliver a solution capable of accurately digitizing multiple raster documents without human intervention. Such digitized logs can then be leveraged to analyze the subsurface including aspects such as lithology, formations, porosity, permeability or fluid properties to support various downstream tasks such as reservoir characterization, well placement, drilling optimization, production optimization or exploration. While particularly valuable in the oil and gas sector, these logs also hold significant potential in emerging applications such as carbon capture and storage (CCS) or geothermal energy. Future work would include fine-tuning our visual language models using knowledge distillation on track headers data to further improve the accuracy of the header metadata extraction step.

Acknowledgments

This work was performed in the frame of the DELFI team in the SLB Montpellier Technology Center. We thank the team for providing us with access to computing infrastructure, both on premise and on the cloud, which allowed us to develop our fully automated digitization pipeline and to evaluate it. We also thank the team at the SLB Pune Technology Center for sharing with us their pre-trained deep learning segmentation and digitization models which we use in our solution after extracting automatically all necessary metadata from the well log raster document.

References

Agrawal, P. et al, 2024. ‘Pixtral 12B’. *arXiv [Cs.CV]. arXiv*. https://doi.org/10.48550/arXiv.2410.07073.

Kamath, A. et al, 2025. ‘Gemma 3 Technical Report’. *arXiv [Cs.CL]. arXiv*. https://doi.org/10.48550/arXiv.2503.19786.

Kwon, W. et al, 2023. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. https://doi.org/10.48550/arXiv.2309.06180.

Otsu, N., 1979. "A Threshold Selection Method from Gray-Level Histograms." *IEEE Transactions on Systems, Man, and Cybernetics*.

Ratcliff, J. W and Metzener, D., July 1988. "Pattern Matching: The Gestalt Approach". *Dr. Dobb's Journal* (46).

Shakeel, M. S., 2024. Conditional Curve Segmentation for Raster Digitization. no. 1, *European Association of Geoscientists & Engineers*, pp. 1–5, doi:https://doi.org/10.3997/2214-4609.202410485.

Smith, R., 2007. "An Overview of the Tesseract OCR Engine." Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR).

Witte, D., Xuan, R., Witte, I., Sep. 2020. *Raster Log Digitization System and Method. U.S. Patent No. 10,776,967 B2*.

Xiao, B. et al, 2023. *Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks*. https://doi.org/10.48550/arXiv.2311.06242.